

Class Notes: Datasets & Data Dependencies

1. Why do you need that feature? (The Problem)

Before Datasets, DAG dependencies were solely based on **time**. A DAG (consumer_dag) could be scheduled to run *after* another DAG (producer_dag) based on time intervals (e.g., @daily). But this had major flaws:

1. **Inefficiency (Wasted Cycles):** The producer_dag might fail or be delayed. The consumer_dag would still run at its scheduled time, process outdated or missing data, and also fail, causing alert noise and wasted resources.
2. **Imprecise:** The producer_dag might finish in 5 minutes or 2 hours. The consumer_dag has no way of knowing. You'd have to schedule the consumer with a large delay, "hoping" the producer is done, which is inefficient.
3. **Complexity for Multiple Producers:** If a consumer DAG depends on data from *multiple* producer DAGs (e.g., a report needs data from a "sales_etl" DAG *and* a "users_etl" DAG), orchestrating this with time-based scheduling was incredibly complex and brittle. You had to rely on complex Sensor logic or external state tracking.

The Need: A paradigm shift from **time-based scheduling** to **data-aware scheduling**. A consumer DAG should only run when the data it depends on has been **updated**.

2. What is a Dataset?

A **Dataset** is a new, first-class concept in Airflow (introduced in version 2.4) that represents a logical data product.

- **It's a URI:** It's essentially a string identifier formatted as a URI (e.g., s3://my-bucket/data/transactions/, postgresql://postgres:5432/my_db/public/my_table, datawarehouse://sales/report).
- **It's not the data itself:** It does not hold the actual data. It is a **metadata object**, a pointer that signifies a location or a name of a dataset.
- **Purpose:** It acts as a trigger. When a task updates a dataset (e.g., writes a file to s3://my-bucket/data/), it notifies the Airflow scheduler. The scheduler then triggers any DAG that depends on that dataset.

How it works:

1. A **producer task** (in a producer DAG) declares that it updates a specific dataset upon completion.
2. A **consumer DAG** is scheduled not on a time schedule, but on the update of one or more datasets.
3. When the producer task completes successfully, it "outlets" a dataset event.
4. The scheduler sees this event and triggers a run of the consumer DAG.

Image Suggestion: A simple diagram showing a box (Producer Task) with an arrow labeled "Updates" pointing to a database icon labeled "Dataset: postgresql://db/schema/table". Another arrow from the dataset icon labeled "Triggers" points to a second box (Consumer DAG).

3. Create the Producer DAG

The Producer DAG is responsible for creating or updating a data asset. Its tasks use the `outlets` parameter to declare which datasets they update.

Example: A DAG that generates a user summary file on S3.

```
from airflow import DAG

from airflow.datasets import Dataset

from airflow.operators.python import PythonOperator

from airflow.providers.amazon.aws.transfers.local_to_s3 import LocalFilesystemToS3Operator

from datetime import datetime

import json


# Define the Dataset URI. This is the key concept.
USER_SUMMARY_DATASET = Dataset("s3://airflow-training-bucket/data/user_summary.json")


def _generate_user_summary():
    """A Python function that generates some summary data and saves it locally."""
    # ... logic to generate summary data ...
    summary_data = {"total_users": 150, "active_users": 120}
    with open('/tmp/user_summary.json', 'w') as f:
        json.dump(summary_data, f)


with DAG(
    dag_id="producer_dag",
    start_date=datetime(2023, 1, 1),
    schedule="@daily",
    catchup=False
) as dag:

    generate_file = PythonOperator(
```

```

    task_id="generate_file",
    python_callable=_generate_user_summary
    # This task does not outlet the dataset yet, it just creates a local file.
)

upload_to_s3 = LocalFilesystemToS3Operator(
    task_id="upload_to_s3",
    filename='/tmp/user_summary.json',
    dest_key='data/user_summary.json',
    dest_bucket='airflow-training-bucket',
    replace=True,
    # THIS is the crucial part.
    # This task outlets the dataset, declaring that upon success,
    # it has updated the data at the specified URI.
    outlets=[USER_SUMMARY_DATASET]
)

generate_file >> upload_to_s3

```

Key Point: The outlets parameter is what creates the data dependency link. The task that *finally* updates the data should be the one that outlets the dataset.

4. Create the Consumer DAG

The Consumer DAG is triggered by updates to one or more datasets, not by time. Notice the schedule parameter.

Example: A DAG that reads the summary file and sends a report.

```

from airflow import DAG
from airflow.datasets import Dataset
from airflow.operators.python import PythonOperator
from airflow.providers.amazon.aws.hooks.s3 import S3Hook
from datetime import datetime
import json

# Define the SAME Dataset URI. This creates the dependency.

```

```

USER_SUMMARY_DATASET = Dataset("s3://airflow-training-bucket/data/user_summary.json")

def _generate_report():
    """Reads the data from S3 and generates a report."""
    s3_hook = S3Hook(aws_conn_id='aws_default')
    file_content = s3_hook.read_key(key='data/user_summary.json', bucket_name='airflow-training-bucket')
    data = json.loads(file_content)

    print(f"REPORT: Total Users: {data['total_users']}, Active Users: {data['active_users']}")
    # In a real scenario, you might send an email or Slack message here.

# The Consumer DAG
with DAG(
    dag_id="consumer_dag",
    start_date=datetime(2023, 1, 1),
    # Schedule is defined by the DATASET, not a time interval.
    # This DAG will run whenever the specified dataset is updated.
    schedule=[USER_SUMMARY_DATASET],
    catchup=False,
    tags=["data-aware"]
) as dag:

    generate_report = PythonOperator(
        task_id="generate_report",
        python_callable=_generate_report
    )

    # No need to define dependencies between tasks here if there's only one.

```

Advanced Scenario: Multiple Datasets

A consumer can depend on multiple datasets. It will only run when *all* listed datasets have been updated at least once since the last consumer run.

```
SALES_DATASET = Dataset("s3://my-bucket/sales/data.parquet")
INVENTORY_DATASET = Dataset("s3://my-bucket/inventory/data.parquet")

with DAG(
    dag_id="combined_report_dag",
    start_date=datetime(2023, 1, 1),
    schedule=[SALES_DATASET, INVENTORY_DATASET], # AND dependency
    catchup=False
) as dag:
    ...
```

5. Track your Datasets with the new view

Airflow 2.4+ introduces a dedicated UI view for monitoring datasets.

- **Access:** Go to **Browse -> Datasets** in the top menu.
- **What it shows:**
 - **URI:** The identifier of the dataset.
 - **Last Update:** The timestamp of the most recent update event from any producer task.
 - **Producing DAGs:** A list of DAGs that have tasks which outlet this dataset.
 - **Consuming DAGs:** A list of DAGs that are scheduled on this dataset.
- **Functionality:** You can see the entire chain of data dependencies at a glance. This is invaluable for data lineage and impact analysis (e.g., "If I change this producer DAG, which reports will be affected?").

Image Suggestion: A screenshot of the "Datasets" list view in the Airflow UI. It should show 2-3 datasets, each with a "Last Update" time and links to producing/consuming DAGs. A second screenshot showing the detail view of one dataset, listing all its producers and consumers.

6. Dataset Limitations

While powerful, Datasets have important limitations to understand:

1. **No Data Content Awareness:** The system only knows that a dataset *was updated*, not *what was changed*. A consumer runs if the producer task succeeded, even if the update was a no-op or wrote the exact same data. It's a data *update* trigger, not a data *change* trigger.
2. **URI Definition is Arbitrary:** The URI is just a string. Airflow does not check its validity or enforce any structure. Consistency is up to the developers. A typo in the URI (e.g., `s3://bucket/` vs. `s3://bucket`) will break the dependency.

3. **Event-Based, Not State-Based:** The scheduler counts update events. If a producer DAG runs 10 times, it will trigger the consumer DAG 10 times, regardless of the state of the actual data file.
4. **Not for Large Data Transfer:** Like XCom, Datasets are metadata triggers. They should **not** be used to pass the actual data payload between DAGs. The data itself should be stored in a persistent storage system (S3, GCS, database).
5. **DAG Structure Required:** You still need a producer DAG to create the data and a consumer DAG to consume it. It doesn't replace the need for well-designed pipelines.

When to Use: Ideal for orchestrating pipelines where DAGs produce and consume well-defined data assets (e.g., tables, files in cloud storage). It moves Airflow from a pure time-based scheduler to a dataflow orchestrator.

When Not to Use: For triggering based on the *content* of data (e.g., if a value in a record exceeds a threshold), you would still use a Sensor within a DAG.